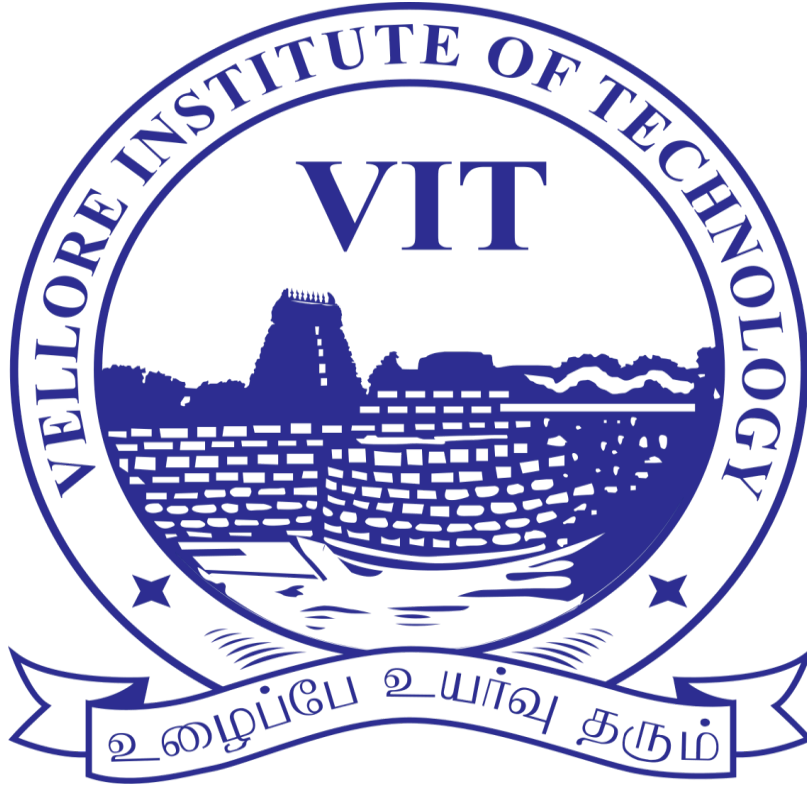




Vellore Institute of Technology, Chennai

Fall Semester(2026-2027)

DA-2 REVIEW

Course: Software Engineering

Course code: BCSE301L

Faculty name: DR. ALOK CHAUHAN

**Team members- RUDRANSH SHARMA(24BAI1510)
ARSHDEEP PASRICHA(24BAI1501)**

a. Design Principles Applied

SafeBite enforces core software engineering principles to ensure architectural maintainability, testability, and strict separation of concerns.

1. Abstraction

- **Application:** Decouples internal database persistence models from external API contracts.
- **Implementation:** Pydantic schemas (`schemas.py`) define explicit Data Transfer Objects (DTOs), preventing raw database models from leaking sensitive attributes (e.g., `password_hash`) to the presentation tier.
- **Why:** This design prevents changes to database models from directly affecting API consumers and ensures sensitive internal fields cannot accidentally be exposed.

```
# backend/app/schemas/schemas.py
class UserOut(UserBase):
    id: int
    created_at: datetime.datetime
    # password_hash is strictly abstracted away from API consumers
    class Config:
        from_attributes = True
```

2. Modularity

- **Application:** Segregates business capabilities into bounded, independently maintainable modules.
- **Implementation:** The backend isolates routing across discrete domain modules (`auth.py`, `restaurants.py`, `inspections.py`, `admin.py`), composed cleanly at the ASGI application root.
- **Why:** Separating each domain into its own router makes individual features easier to modify, test, and extend without introducing changes into unrelated functionality.

```
# backend/app/main.py
app.include_router(auth.router, prefix="/api")
app.include_router(restaurants.router, prefix="/api")
app.include_router(admin.router, prefix="/api")
app.include_router(inspections.router, prefix="/api")
```

3. Cohesion

- **Application:** Maximizes functional cohesion by binding only strictly related logic within a single module.
- **Implementation:** `security.py` concentrates exclusively on cryptographic hashing and JWT token lifecycles, containing zero database or HTTP-routing logic.
- **Why:** Keeping security operations isolated prevents authentication logic from becoming coupled with database or routing concerns, making security-related changes easier to implement and test.

```
# backend/app/core/security.py
def get_password_hash(password: str) -> str:
    # High functional cohesion: strictly handles cryptographic
operations
    salt = bcrypt.gensalt()
    return bcrypt.hashpw(password.encode("utf-8"),
salt).decode("utf-8")
```

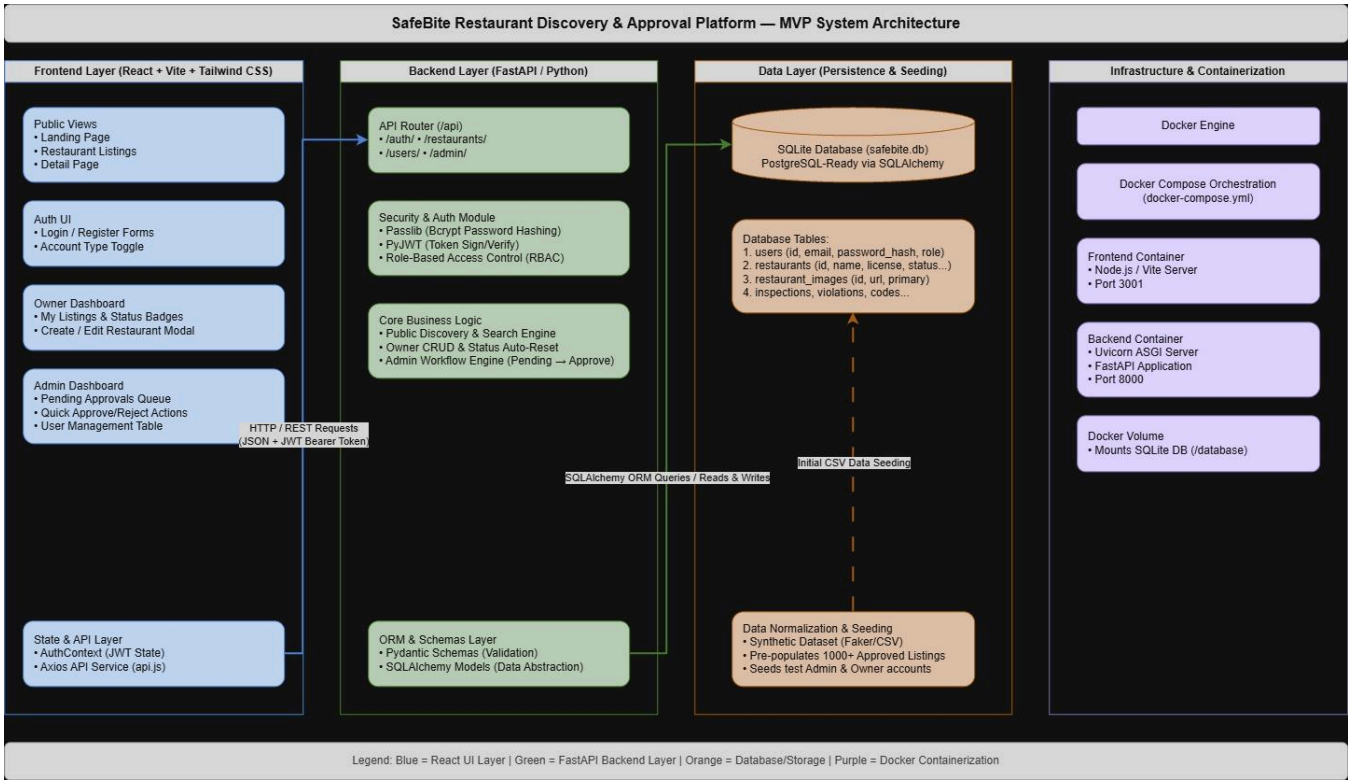
4. Low Coupling

- **Application:** Eliminates hardcoded dependencies through Inversion of Control (Dependency Injection).
- **Implementation:** Route handlers receive database sessions and authenticated user sessions via FastAPI's `Depends()` mechanism, avoiding direct instantiation.
- **Why:** Dependency injection allows database and authentication mechanisms to be changed independently of route handlers, reducing the impact of infrastructure changes on business logic.

```
# backend/app/api/auth.py
def get_current_user(token: str = Depends(oauth2_scheme), db:
Session = Depends(get_db)):
    # Decoupled: Endpoint does not manage session initialization or
teardown
    payload = decode_access_token(token)
    return db.query(User).filter(User.email ==
payload.get("email")).first()
```

b. High-Level Architecture

SafeBite utilizes a **Containerized Multi-Tier Client-Server Architecture** running decoupled Single-Page Application (SPA) and REST API tiers across isolated Docker services.



Why this architecture was chosen:

The multi-tier architecture was selected to isolate presentation, application, persistence, and infrastructure concerns. This prevents changes in one layer, such as replacing the database or modifying the UI, from requiring changes throughout the system. Docker further isolates the frontend and backend environments, making the system easier to deploy consistently and maintain.

1. Architectural Decomposition

Tier / Layer	Technology Stack	Core Responsibilities
Presentation Tier	React 18, Vite, Tailwind CSS, Axios, React Router v6	Client-side routing, interactive inspection timelines, responsive filtering, and JWT session state (AuthContext).
Application Tier	FastAPI (Python 3.12), Uvicorn ASGI, Pydantic, PyJWT	Asynchronous request handling, RBAC enforcement, schema validation, safety score calculation, and admin verification engines.

Tier / Layer	Technology Stack	Core Responsibilities
Persistence Tier	SQLite (<code>safebite.db</code>), SQLAlchemy ORM	Relational data persistence (Users, Restaurants, Inspections, Violations), relationship cascades, and synthetic dataset seeding.
Infrastructure Tier	Docker Engine, Docker Compose (<code>safebite-network</code>)	Service isolation (Frontend: 3001, Backend: 8000), internal bridge networking, and persistent volume mounting.

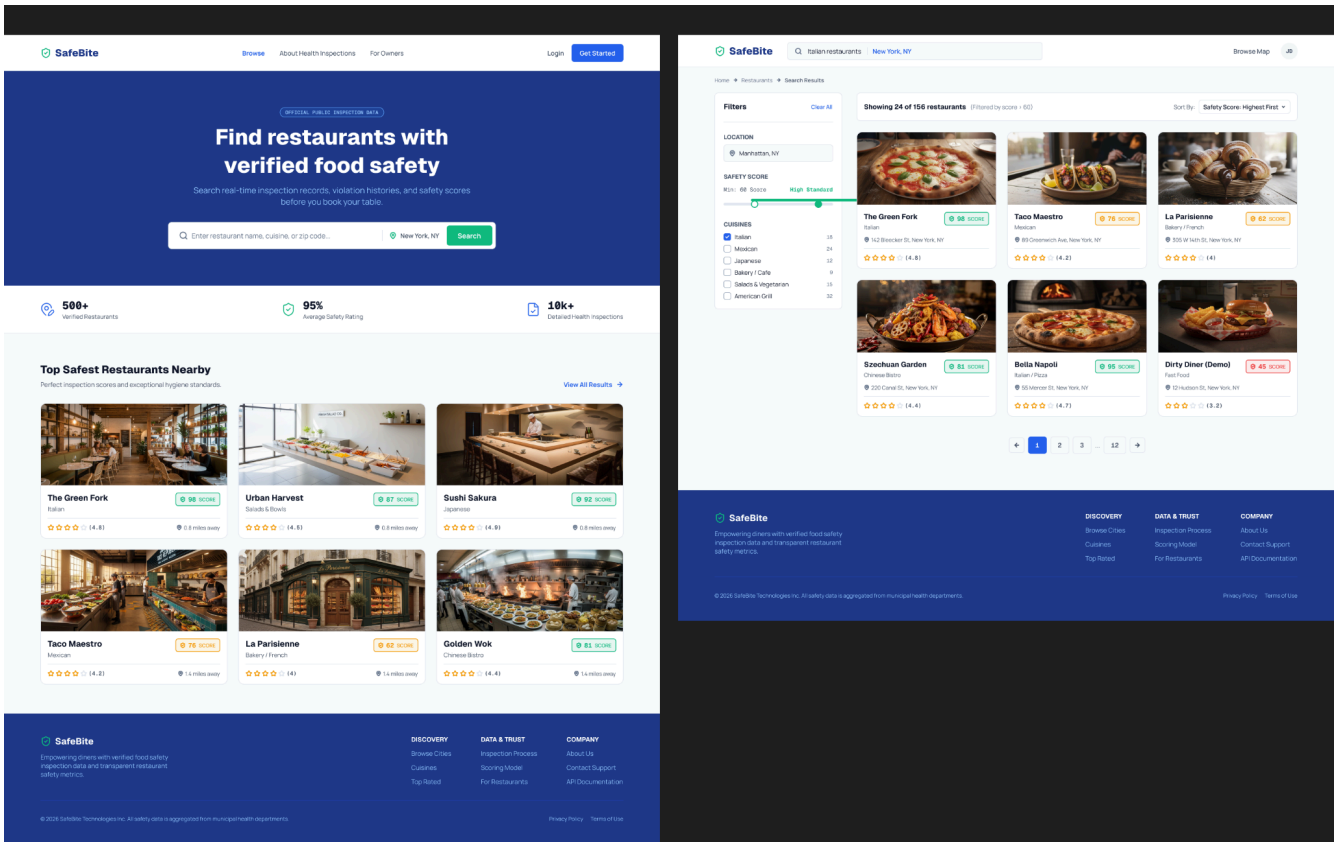
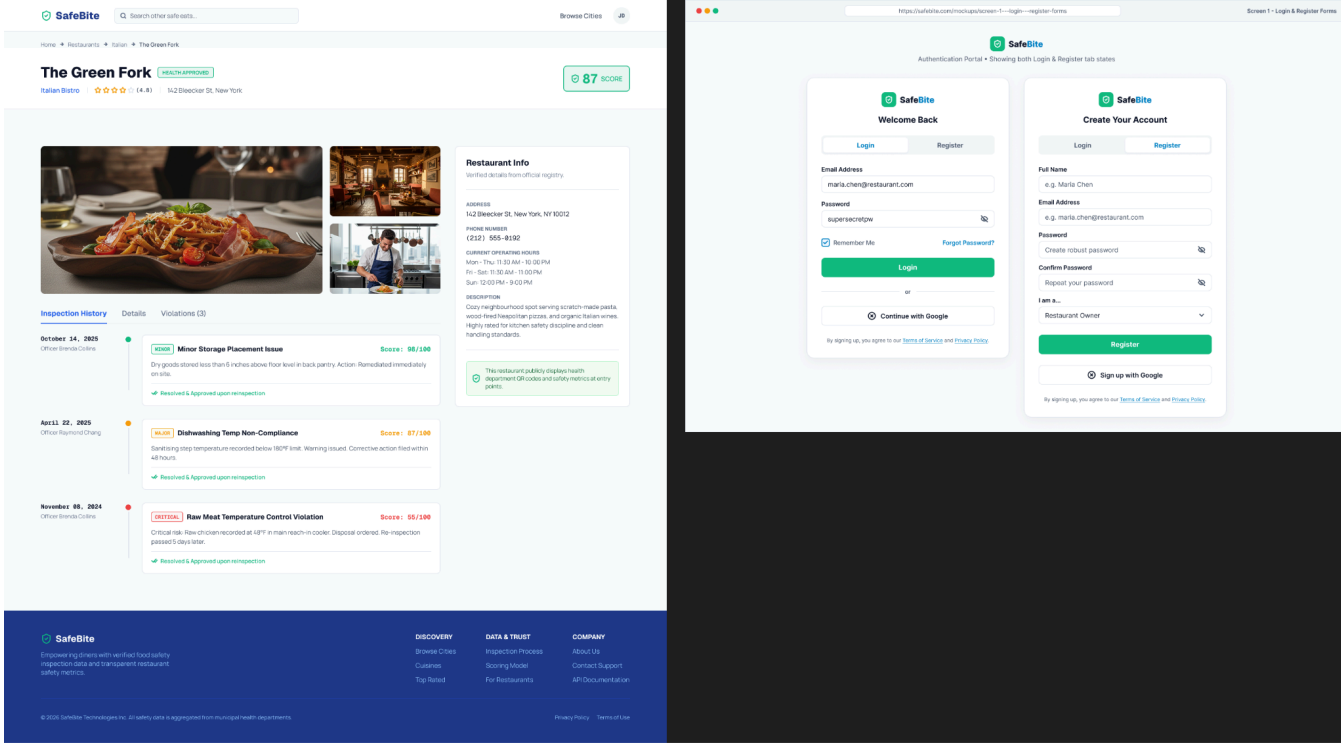
2. Transaction & Authentication Flow

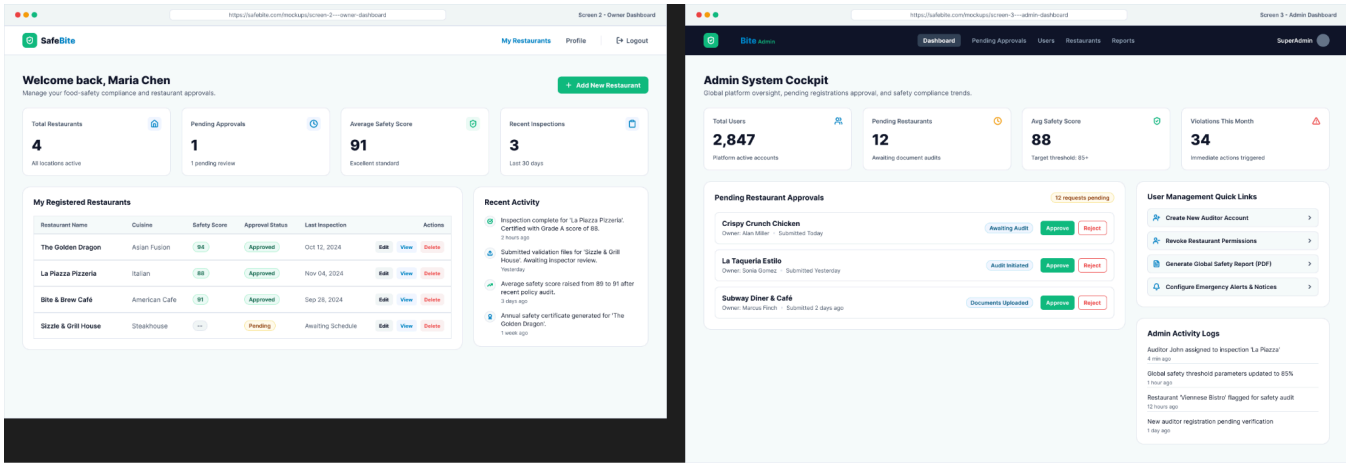
- **Data Flow:** User Interaction → React Component → Axios Client (`api.js`) → HTTP/REST Request (Port 8000) → FastAPI Router → Dependency Injection (Auth & Session) → SQLAlchemy ORM → SQLite Query → Pydantic Serialization → JSON Response to UI.
- **Auth Pipeline:** POST `/api/auth/login` → Bcrypt Hash Verification → Signed HS256 JWT Generated → Stored in Browser LocalStorage → Transmitted via `'Authorization: Bearer <token>'` → Verified per-request by FastAPI RBAC guards.

3. Architecture Rationale

- 1. Strict Separation of Concerns:** Isolating the UI from API logic allows independent frontend and backend deployments, enabling future mobile clients (React Native) to consume the identical REST API.
- 2. Asynchronous Throughput:** FastAPI's ASGI architecture supports efficient handling of concurrent I/O-bound requests, making it suitable for responsive restaurant search and API operations.
- 3. Stateless Scalability:** Token-based authentication eliminates server-side session memory, allowing the backend container to scale horizontally behind a load balancer without sticky sessions.
- 4. Environment Parity:** Multi-container Docker Compose definitions eliminate environment discrepancies across development, evaluation, and staging environments.
- 5. Database Agnosticism:** SQLAlchemy ORM abstracts SQL dialects, enabling instant transition from SQLite (local development) to PostgreSQL (production) solely via environment variable configuration.

c. User Interface Design (Updated 6 Figma screens)





1. Screen Inventory & Component Breakdown

Screen	View / Route	Key Interface Elements	User Stories Addressed
1. Login / Sign Up	/auth	Dual-card portal showing Login and Register states, password visibility toggle, role dropdown (Restaurant Owner / Customer), and Google OAuth action.	Epic 1: Stories 1, 2
2. Home Page	/	Deep blue hero banner, official public inspection data pill, dual-input search bar (cuisine + location New York, NY), 3 trust metrics ribbon, and top safest restaurant cards.	Epic 2, 6: Stories 4, 6, 21
3. Restaurant Listing	/restaurants	Sticky left sidebar (location input, interactive Safety Score Slider Min: 60, cuisine counts), sorting dropdown (Highest Safety Score), 3x2 card grid, and pagination.	Epic 2: Stories 4, 5, 6, 22
4. Restaurant Detail	/restaurants/:id	The Green Fork dossier with HEALTH APPROVED tag, prominent 87 SCORE green badge, 3-photo food gallery, operating hours card, and chronological inspection history with color-coded severity dots.	Epic 3: Stories 7, 8, 9, 10, 11

Screen	View / Route	Key Interface Elements	User Stories Addressed
5. Owner Dashboard	/owner /dashb oard	Metric summary cards (Total Restaurants: 4, Pending: 1, Avg Score: 91), + Add New Restaurant CTA, registered restaurants table with status pills (Approved / Pending), and recent audit activity feed.	Epic 4: Stories 13, 14, 15, 16
6. Admin Dashboard	/admin /dashb oard	Dark cockpit navbar, system metric overview (2,847 Users, 12 Pending), pending approval cards with 1-click Approve (green) / Reject (red) actions, and user administration quick links.	Epic 5: Stories 17, 18, 19, 20

2. UI Usability Evaluated by Core Principles

1. Consistency: Standardized Tailwind CSS design tokens establish uniform color semantics across all screens: Emerald Green (#10B981) for safety/approval, Deep Blue (#1E3A8A) for brand elements, Amber (#F59E0B) for pending audits, and Crimson (#EF4444) for critical violations.

2. Clarity and Simplicity: Uncluttered layouts prioritize high-contrast primary actions (Approve, Register, Search). Complex statutory health records are translated into readable tags (e.g., *"Minor Storage Placement Issue — Corrected on Site"*).

3. Feedback: The system provides deterministic visual feedback: green/red input field borders during validation, active slider fill tracks, and instant status updates upon administrative approval.

4. User Control and Freedom: Modal forms include top-right close icons (✕) and cancel buttons. The listing view provides a single-click *"Clear All"* filter reset, and breadcrumb trails permit effortless navigation back up page hierarchies.

5. Reduce Memory Load: Universal traffic-light color conventions (Green = Safe, Yellow = Warning, Red = Danger) and circular gauge score dials allow users to assess hygiene instantly without reading raw checklists.

6. Accessibility and Inclusivity: All text satisfies WCAG 2.1 AA contrast ratios (minimum 4.5:1). Color is never used as the sole indicator: safety scores always couple color badges with explicit textual labels (e.g., "98 *SCORE* — *Excellent*").

7. Error Prevention and Recovery: Submit buttons remain disabled until required formats (e.g., valid email and license numbers) pass validation. High-impact actions (deleting a listing or rejecting an application) require confirmation dialogues.

d. Design Decisions & Technical Rationale

1. FastAPI over Flask / Django

- **Decision:** Adopt FastAPI (ASGI) for the backend.
- **Why:** Native Pydantic validation and asynchronous support reduce boilerplate and improve API responsiveness.
- **Trade-off:** Requires strict type annotations, but provides automatic API documentation.

2. Stateless JWT Authentication over Server Sessions

- **Decision:** Use signed JWTs (HS256) instead of server-side sessions.
- **Why:** Keeps authentication stateless and allows the backend to scale without maintaining session storage.
- **Trade-off:** Early token revocation is difficult; controlled token expiration mitigates this.

3. SQLAlchemy ORM over Raw SQL

- **Decision:** Use SQLAlchemy ORM for database persistence.
- **Why:** Provides parameterized queries, clean relationship mapping, and database portability between SQLite and PostgreSQL.
- **Trade-off:** Slightly more overhead than raw SQL, but improves maintainability.

4. Admin Approval with Critical Edit Invalidation

- **Decision:** New listings remain Pending, and critical edits reset approved listings to Pending.
- **Why:** Prevents unauthorized changes to important information such as restaurant names and license details.
- **Trade-off:** Adds approval delay, but protects consumer trust.

5. Client-Side API Facade (`api.js`)

- **Decision:** Centralize Axios calls in domain services such as `restaurantApi`, `adminApi`, and `userApi`.
- **Why:** Centralizes authentication and API configuration, so backend changes require fewer UI modifications.
- **Trade-off:** Adds a small abstraction layer, but reduces duplication and improves maintainability.

Conclusion: Maintainability & Future Changes (Software Evolution)

SafeBite's architecture is engineered around the **Open/Closed Principle (OCP)** and a decoupled multi-tier topology, ensuring the platform can evolve gracefully over its lifecycle across adaptive, perfective, and preventive maintenance.

High Maintainability:

- **Domain-Isolated Modules:** Routing and business logic are segregated by domain (`auth`, `restaurants`, `inspections`). Adding features (e.g., consumer reviews) follows the Open/Closed Principle—new routes plug in without touching existing code.
- **Database Portability:** Using SQLAlchemy ORM abstracts SQL syntax, allowing an instant transition from SQLite to PostgreSQL by simply updating the `DATABASE_URL` environment variable.
- **Centralized API Contracts:** The frontend `api.js` service facade isolates network calls, ensuring backend URL or schema changes require updates in only one file rather than across every UI component.

Future Enhancements:

- **Cross-Platform Expansion:** The stateless, JWT-based REST API is fully client-agnostic, enabling native iOS/Android (React Native/Flutter) apps to consume the exact same backend.
- **Government API & IoT Sync:** Pluggable Pydantic validation schemas allow future automated data synchronization directly with official municipal health databases (FSSAI/FDA) and smart kitchen temperature sensors.
- **Asynchronous Scaling:** Heavy workloads (bulk inspection imports and report generation) can be offloaded to background Celery/Redis queues without blocking core user searches.